

DEEP LEARNING PROJECT REPORT

Robustness Evaluation of Deep Learning-Based Intrusion Detection Systems under Constrained Black-Box Adversarial Attacks and Practical Defense Mechanisms

Deep Learning — Course Project Report

Instructor: Dr. Mehwish Awan

Group Members

Abeha Hussain (454609)

Ahmad Hassan Tanveer (457627)

Muhammad Waleed (455706)

Abstract

Network Intrusion Detection Systems (IDS) built on deep learning have demonstrated near-perfect accuracy under clean traffic conditions. However, their resilience under adversarial manipulation remains poorly understood, particularly in realistic black-box threat scenarios where an attacker has no access to model internals. This project designs and evaluates a complete adversarial machine learning pipeline applied to network IDS. We train two deep learning target models a one-dimensional **Convolutional Neural Network (CNN)** and a **Long Short-Term Memory (LSTM)** network on the **NSL-KDD** benchmark dataset. A **surrogate Multi-Layer Perceptron (MLP)** is trained exclusively on input–output pairs obtained by querying the target CNN under black-box conditions (hard labels only). **Constrained Projected Gradient Descent (PGD)** adversarial examples are crafted on the surrogate and transferred to both target models. Two defense strategies are evaluated: feature-space clipping (input preprocessing) and adversarial training (data augmentation with synthesized adversarial examples). Results show that the CNN and LSTM both achieve **99.42% F1** on clean traffic. Surrogate-based black-box adversarial examples achieve a **4.37% Attack Success Rate (ASR)** against the undefended CNN. Clip defense provides no additional benefit over the raw model for this attack budget, while adversarial training reduces ASR dramatically to **0.13% a 33× reduction** without sacrificing clean detection performance (**F1 remains 99.48%**). Surrogate–target fidelity reaches **97.59%**, validating the black-box threat model. These findings highlight that while DL-based IDS are inherently robust to mild constrained perturbations, adversarial training is a highly effective and computationally tractable defense.

Keywords: *Intrusion Detection Systems, Deep Learning, Adversarial Machine Learning, Black-Box Attacks, Convolutional Neural Networks, LSTM, PGD, Adversarial Training, NSL-KDD*

1. Introduction

1.1 Background and Motivation

Network Intrusion Detection Systems (IDS) are a cornerstone of modern cybersecurity infrastructure. Traditionally, IDS operated through signature matching comparing observed network traffic against a database of known attack patterns. While effective against catalogued threats, signature-based methods are inherently unable to detect novel or zero-day attacks. The explosion of internet traffic volume and the increasing sophistication of adversarial actors have driven a paradigm shift toward Machine Learning (ML) and, more recently, Deep Learning (DL) based IDS.

Deep learning models including Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) such as Long Short-Term Memory (LSTM) networks offer compelling advantages for IDS: they learn complex, hierarchical representations of network traffic directly from raw features, generalize to unseen attack patterns, and can operate at line rate with appropriate hardware acceleration. DL-based IDS routinely achieve detection accuracies exceeding 99% on benchmark datasets such as NSL-KDD and CICIDS2017.

However, the very property that makes deep learning powerful its reliance on learned statistical patterns also introduces a critical vulnerability. In 2014, Goodfellow et al. demonstrated that deep neural networks can be fooled by imperceptibly small, carefully crafted perturbations to their inputs, known as adversarial examples. An adversary who understands this property can systematically construct network traffic that appears malicious to human analysts but is classified as benign by a DL-based IDS, effectively evading detection.

What makes this threat particularly concerning in the IDS domain is that real-world attackers typically do not have access to the IDS model's weights, architecture, or training data they can only observe the system's outputs (block/allow decisions). This is the black-box threat model, and it is the most practically relevant scenario for network security. Yet most existing adversarial IDS research evaluates attacks under white-box conditions (full model access), which overstates real-world risk. This project directly addresses this gap.

1.2 Project Scope

This project implements and evaluates a complete adversarial machine learning pipeline for DL-based IDS, encompassing:

- Training of two deep learning target IDS models (**CNN and LSTM**) on the **NSL-KDD dataset**.
- Simulation of realistic black-box adversarial attacks using a surrogate model trained exclusively on queried input–output pairs.
- **Constrained Projected Gradient Descent (PGD)** attacks that respect the semantic validity of network traffic features.
- Evaluation of two practical defense strategies: input preprocessing (feature clipping) and adversarial training.
- A comprehensive empirical analysis including transferability, epsilon sensitivity, t-SNE visualization of feature space shifts, and ROC analysis.

The remainder of this report is organized as follows. Section 2 reviews relevant literature. Section 3 describes the NSL-KDD dataset and preprocessing. Section 4 details the methodology. Section 5 presents the deep learning architectures with explicit justification of DL design choices. Section 6 reports experimental results with analysis. Section 7 concludes with key findings and future directions.

2. Literature Review

2.1 Deep Learning for Intrusion Detection

The application of deep learning to network intrusion detection has been extensively studied. **Javaid et al. (2016)** were among the first to apply deep autoencoders to IDS, demonstrating superior generalization over traditional ML classifiers on NSL-KDD. **Kim et al. (2016)** showed that LSTM networks are particularly effective at capturing temporal dependencies in sequential network flows, improving detection of low-and-slow attacks that evade stateless classifiers. **Wang et al. (2017)** applied CNNs directly to raw packet payloads converted to pixel-like byte grids, achieving high detection rates without feature engineering.

More recent work has explored ensemble approaches. **Tang et al. (2020)** combined CNN and LSTM components in a parallel architecture, with the CNN extracting local spatial patterns from feature vectors while the LSTM captures inter-feature temporal dependencies an architecture related to the parallel target model

design used in this project. The UNSW-NB15 and CICIDS2017 datasets have largely supplanted **KDD Cup'99** as evaluation benchmarks, but NSL-KDD remains a widely accepted standard for comparative evaluation due to its removal of redundant records.

2.2 Adversarial Machine Learning

The field of adversarial machine learning was formalized by **Goodfellow et al. (2014)** with the introduction of the **Fast Gradient Sign Method (FGSM)**. **Madry et al. (2018)** extended this to the more powerful Projected Gradient Descent (PGD) attack, which iterates FGSM steps with projection back onto an L_∞ epsilon ball and has become the de facto standard for evaluating model robustness. **Carlini and Wagner (2017)** introduced the C&W attack, which finds the minimum-norm perturbation that causes misclassification, and showed that many defenses are ineffective against it.

A critical development for this project is the transferability property of adversarial examples. **Papernot et al. (2017)** demonstrated that adversarial examples crafted on one model (the surrogate) frequently transfer to other models with different architectures trained on different data enabling black-box attacks. Their Jacobian-based Augmentation method synthesizes additional training data for the surrogate by exploiting gradient information, but this project uses a simpler and more realistic approach: training the surrogate purely on queried hard labels.

2.3 Adversarial Attacks on IDS

Applying adversarial ML to IDS introduces a unique challenge absent in image classification: feature-space constraints. Network features are not independent pixels; they encode protocol semantics. For example, the flag field in a TCP connection is categorical (SYN, ACK, FIN, RST, etc.) and cannot take intermediate floating-point values. The `src_bytes` and `dst_bytes` features are non-negative integers. Violating these constraints produces adversarial examples that are technically adversarial in feature space but correspond to invalid network traffic meaningless in a real deployment.

Rosenberg et al. (2021) proposed constrained adversarial attacks on IDS that respect actionability of features only perturbing features that an attacker can realistically manipulate. **Lin et al. (2022)** further studied the problem under black-box conditions with hard-label queries only. **Roshan and Zafar (2024)**, the primary reference for this project, provide a comprehensive framework for evaluating constrained black-box adversarial robustness of DL-based IDS, which this project implements and extends with additional analysis.

2.4 Defense Mechanisms

Adversarial training (**Madry et al., 2018**) augments the training dataset with adversarial examples generated on the model being trained, making the decision boundary more robust to perturbations. Despite its computational cost, adversarial training remains one of the few defenses that provides certified robustness guarantees under certain conditions. Input preprocessing defenses such as feature squeezing, smoothing, and clipping attempt to remove adversarial perturbations before inference without retraining. **Xu et al. (2018)** showed that simple feature squeezing can detect adversarial examples in image classification, but its effectiveness is limited for tabular feature spaces like network traffic where perturbations are subtler.

3. Dataset

3.1 NSL-KDD Overview

The NSL-KDD dataset is a refined version of the **KDD Cup 1999 dataset**, addressing two critical issues in the original: (1) elimination of duplicate records that caused classifiers to be biased toward high-frequency attack types, and (2) selection of records from KDD'99 such that the number of records from each difficulty level is inversely proportional to the percentage of records in the original dataset. NSL-KDD contains **125,973 training records** and **22,544 test records**, representing **41 features** derived from **network connection metadata**.

Each record describes a single network connection and is labeled as either normal traffic or one of four attack categories: Denial of Service (DoS), Probe (reconnaissance), Remote-to-Local (R2L), and User-to-Root (U2R). This project uses binary classification: normal (0) versus attack (1), aligning with the most common IDS deployment scenario. The dataset exhibits a near-balanced class distribution after preprocessing, with **46.5% attack records**, which avoids class imbalance issues.

3.2 Feature Description

The 41 features can be categorized into four groups:

- **Basic connection features (9):** duration, protocol_type, service, flag, src_bytes, dst_bytes, land, wrong_fragment, urgent
- **Content features (13):** hot, num_failed_logins, logged_in, num_compromised, root_shell, su_attempted, num_root, num_file_creations, num_shells, num_access_files, num_outbound_cmds, is_host_login, is_guest_login
- **Traffic features (9):** count, srv_count, error_rate, srv_error_rate, rerror_rate, srv_rerror_rate, same_srv_rate, diff_srv_rate, srv_diff_host_rate
- **Host-based features (10):** dst_host_count, dst_host_srv_count, dst_host_same_srv_rate, dst_host_diff_srv_rate, dst_host_same_src_port_rate, dst_host_srv_diff_host_rate, dst_host_error_rate, dst_host_srv_error_rate, dst_host_rerror_rate, dst_host_srv_rerror_rate

After one-hot encoding of three categorical features (**protocol_type: 3 values; service: 70 values; flag: 11 values**), the feature dimensionality expands from **41 to 122 features**. The label and difficulty-level columns are excluded from the feature matrix, yielding an input dimension of 122.

3.3 Three-Way Stratified Data Split

A critical design decision in this project is the three-way partition of the data, motivated directly by the black-box threat model. Unlike a conventional two-way train/test split, three disjoint sets are required:

Partition	Size	Proportion	Purpose
Target Train	75,583	60%	Train CNN and LSTM target IDS models
Query Pool	25,195	20%	Attacker probes target here; (input, predicted-label) pairs used to train surrogate

Test Set	25,195	20%	Held out; evaluates all conditions (clean, adversarial, defended)
-----------------	--------	-----	---

All splits are stratified by class label, preserving the **53.5% normal / 46.5% attack ratio** in each partition. The query pool is the most novel element of this design: it represents the only data the attacker has access to. The attacker queries the target model with these **25,195 samples** and collects the model's binary predictions no probability scores, no gradients, no model internals. This constitutes the strictest black-box assumption.

3.4 Preprocessing Pipeline

The preprocessing pipeline is applied in the following order, fitting parameters exclusively on the target training split and transforming the query pool and test set using the fitted parameters (preventing data leakage):

1. **Categorical encoding:** protocol_type, service, and flag are one-hot encoded. The encoding vocabulary is fixed from the training split.
2. **Numerical scaling:** All **32** continuous numerical features are scaled using RobustScaler, which is median- and IQR-based and therefore resilient to outliers common in network traffic data.
3. **Deduplication and NA removal:** Applied only during training data preparation; test/query sets are not deduplicated.
4. **Binary label creation:** outcome column mapped to **0 (normal)** or **1 (attack)**.

After preprocessing, **32 continuous features** are identified as perturb-able for the adversarial attack phase. The remaining **90 features (6 binary flags + 84 one-hot categorical indicators)** are frozen the PGD attack is explicitly constrained to never modify them.

4. Methodology

The project implements a ten-step adversarial evaluation pipeline. Each step is described below.

4.1 Target IDS Model Training

Two deep learning classifiers are trained independently on the target training split (**75,583 samples**). Both use binary cross-entropy loss, the Adam optimizer with an **initial learning rate of 0.001**, **batch size 128**, and **early stopping with patience 7 (restoring best weights)**. **ReduceLROnPlateau** halves the learning rate if validation loss plateaus for **3 consecutive epochs**, with a **minimum learning rate of 10^{-6}** . A **15% validation split** is carved from the target training data during training.

The **CNN target** converges in **27 epochs**; the **LSTM**, in **50 epochs** due to its more complex recurrent dynamics.

4.2 Black-Box Querying

The attacker sends all 25,195 query pool samples to the CNN target model and records only the hard binary prediction (0 or 1) for each sample. No probability scores, logits, gradients, or model parameters are observed. This constitutes a decision-based black-box attack setup the most restrictive and realistic threat model.

The resulting 25,195 (input, label) pairs where labels are the target's predictions, not the ground truth form the surrogate training set. The intentional inclusion of the target's mispredictions in this set is important: the surrogate must learn to mimic the target's decision boundary, not the true data distribution.

4.3 Surrogate Model Training

An MLP surrogate with three hidden layers (**128 → 64 → 32 neurons, ReLU activations, batch normalization, 25% dropout**) is trained on the 25,195 queried pairs. The surrogate architecture is deliberately different from the CNN/LSTM targets: it uses no convolutional or recurrent layers. This architectural difference tests whether adversarial examples transfer across architectural boundaries a stronger claim than same-architecture transfer.

Surrogate–target fidelity is measured as the agreement between the surrogate's predictions and the CNN target's predictions on the held-out test set. A fidelity of **97.59% is achieved**, indicating that the surrogate successfully approximates the target's decision boundary from black-box observations alone.

4.4 Constrained Projected Gradient Descent Attack

Adversarial examples are generated using PGD with the following configuration:

Hyperparameter	Value
Perturbation budget (ϵ)	0.15 (L^∞ norm)
Step size (α)	0.01
Iterations	50
Random start	Yes
Target class	0 (normal) — evasion attack

Two projections are applied after each PGD step:

- **L^∞ projection:** $\delta = \text{clip}(X_{\text{adv}} - X_{\text{orig}}, -\epsilon, +\epsilon)$, applied only to perturb-able features.
- **Feature-bound projection:** each perturb-able feature is clipped to **[min_train, max_train]**, ensuring values remain within the range observed during training.

The attack is run on the surrogate (white-box), and the resulting adversarial examples are transferred to the CNN and LSTM targets without further optimization this is the black-box transfer step.

4.5 Defense Strategy 1: Input Preprocessing (Feature Clipping)

The simplest defense clips each continuous feature of an input sample to its observed training-set range **[min_train, max_train]** before passing it to the classifier. The rationale is that adversarial perturbations may push feature values outside their natural distribution; clipping back removes a portion of the adversarial signal. This defense requires no retraining and adds negligible inference overhead.

4.6 Defense Strategy 2: Adversarial Training

Adversarial training augments the target training set with adversarial examples and retrain the model from scratch. Concretely:

5. **30% of the attack training samples (10,553 samples)** are selected randomly.
6. PGD is run on the CNN target (white-box, since the defender owns the model) with $\epsilon = 0.10$, $\alpha = 0.01$, **20 steps**, to generate adversarial versions of these samples.

- The augmented dataset (**75,583 original + 10,553 adversarial = 86,136 samples**) is used to retrain a fresh CNN from scratch with the same architecture and training procedure.

The smaller $\epsilon = 0.10$ used during adversarial training (**vs. 0.15 for evaluation**) is intentional: it trains robustness to a range of perturbation budgets, not just the evaluation budget.

5. Deep Learning Models and Justification

This section describes each model architecture in detail and justifies how each component applies core deep learning principles to the IDS domain. The relevance of deep learning rather than classical ML is argued explicitly.

5.1 Why Deep Learning for IDS?

Classical machine learning approaches such as Random Forests and SVMs require manual feature selection and struggle to capture high-order interactions among the **122-dimensional feature vectors** in NSL-KDD. Deep learning addresses this through hierarchical representation learning: each layer transforms the input into progressively more abstract representations, automatically discovering which feature combinations are discriminative for classification.

The three specific DL properties that make these models suitable for IDS are:

- Automatic feature learning:** DL models learn to identify which combinations of network statistics (e.g., high `error_rate` combined with low `same_srv_rate`) indicate a SYN flood attack, without explicit feature engineering.
- Non-linear decision boundaries:** A three-layer MLP with ReLU activations has universal approximation capability, but CNN and LSTM architectures impose structural inductive biases (locality and sequentiality) that match the structure of the feature space.
- Scalability:** Once trained, a forward pass through a 3-layer CNN is $O(1)$ in the number of features far more efficient than kernel SVM inference on 122-dimensional vectors.

5.2 Target Model 1: One-Dimensional CNN

The CNN treats the 122-feature vector as a 1D "sequence" of length 122 (with channel depth 1 after the Reshape layer). This is a well-established technique in tabular feature learning: convolutional filters slide over groups of adjacent features, capturing local feature interactions that may correspond to semantically related groups (e.g., the four `dst_host_*` error rate features tend to co-vary during DDoS attacks).

5.2.1 Architecture

Layer	Output Shape	Parameters	Purpose
Input	(122,)	—	122-dimensional feature vector
Reshape	(122, 1)	0	Add channel dimension for Conv1D
Conv1D (64, k=3)	(122, 64)	256	Capture local feature interactions

BatchNorm + Dropout(0.3)	(122, 64)	256	Regularization and training stability
MaxPooling1D (2)	(61, 64)	0	Reduce spatial resolution, improve invariance
Conv1D (128, k=3)	(61, 128)	24,704	Higher-order feature interactions
BatchNorm + Dropout(0.3)	(61, 128)	512	Regularization
MaxPooling1D (2)	(30, 128)	0	Dimension reduction
Flatten	(3840,)	0	Prepare for dense layers
Dense (64, ReLU)	(64,)	245,824	Global feature combination
Dropout (0.3)	(64,)	0	Prevent overfitting
Dense (1, Sigmoid)	(1,)	65	Binary classification output

5.2.2 DL Concepts Applied

This architecture applies the following deep learning concepts:

- **Convolutional feature extraction:** Weight sharing across the feature sequence reduces parameters and improves generalization compared to a fully-connected approach over all 122 features.
- **Hierarchical representation:** Two conv blocks build increasingly abstract representations the first block detects low-level feature co-occurrence, the second detects patterns of patterns.
- **Batch Normalization:** Applied after each convolutional layer, normalizing activations across the batch dimension. This accelerates convergence, reduces sensitivity to initialization, and acts as a mild regularizer. During adversarial training, batch norm also has the effect of making the model less sensitive to feature-scale perturbations.
- **Dropout regularization:** With a **30% dropout rate at three locations**, the model is prevented from co-adapting neurons each neuron must independently capture useful information.
- **Adaptive learning rate scheduling:** **ReduceLROnPlateau** monitors validation loss and halves the learning rate upon plateau, enabling fine-grained convergence that a fixed rate cannot achieve.

5.3 Target Model 2: LSTM

The LSTM treats the feature vector as a sequence of 122 time steps, each of dimension 1. While network features are not inherently temporal within a single connection record, the sequential processing allows the LSTM to model dependencies between features at different positions in the vector analogous to how attention mechanisms relate non-adjacent tokens in text.

5.3.1 Architecture

Input (122,) → Reshape (122, 1) → LSTM (64 units, return_sequences=True) → Dropout (0.3) → LSTM (32 units) → Dropout (0.3) → Dense (32, ReLU) → Dense (1, Sigmoid).

5.3.2 DL Concepts Applied

- **Long-term dependency modeling:** LSTM gating mechanisms (input gate, forget gate, output gate) allow the model to selectively retain and discard information across feature positions. Features that

only indicate attacks when combined with distant features (e.g., `serror_rate` high AND `dst_host_count` high) can be captured through the hidden state.

- **Stacked recurrence:** Two LSTM layers create a deeper recurrent representation the first layer outputs a feature-wise sequence that the second layer further abstracts.
- **Gradient flow through time:** Unlike vanilla RNNs, LSTMs have gated memory cells that allow gradient flow over long sequences without vanishing/exploding gradient problems critical for processing 122 feature steps.

5.4 Surrogate Model: MLP

The surrogate is a **three-layer MLP (128 → 64 → 32 hidden neurons)** with **batch normalization** and **25% dropout**. Its role is to approximate the CNN target's decision boundary using only black-box queries. The MLP is chosen as the surrogate architecture because:

- It has no structural bias toward local or sequential patterns, forcing it to learn a global approximation of the target's boundary.
- Its simpler architecture means that if adversarial examples transfer from MLP to CNN/LSTM, the transferability is genuinely meaningful it reflects a shared vulnerability in the feature space rather than a shared architectural inductive bias.
- MLP training converges quickly on the 25,195-sample query set, making repeated query-and-train cycles feasible.

5.5 Training Regularization and Optimization

All models share a common training framework that embodies best practices in modern deep learning:

- **Early stopping (patience=7):** Monitors validation loss and restores the best checkpoint, preventing overfitting without requiring a predetermined epoch count.
- **Adam optimizer:** Combines momentum (first moment) and adaptive gradient scaling (second moment), making it robust to sparse gradients and varying loss landscape curvatures across the 122 input dimensions.
- **Binary cross-entropy loss:** Appropriate for binary classification; minimized value implies maximum log-likelihood of the Bernoulli distribution parameterized by the sigmoid output.
- **Validation split (15% of training data):** Provides an unbiased estimate of generalization during training, used by early stopping and learning rate scheduling.

6. Results and Analysis

6.1 Target Model Performance (Clean Test Set)

Both target models achieve near-identical and exceptionally high performance on the clean test set of 25,195 samples. Table 2 summarizes the results.

Model	Accuracy	Precision	Recall	F1 Score
CNN Target	99.46%	99.76%	99.09%	99.42%
LSTM Target	99.46%	99.60%	99.25%	99.42%

Surrogate (MLP)	97.37%	99.33%	94.99%	97.11%
Robust CNN (Adv. Training)	99.51%	99.52%	99.43%	99.48%

Table 2: Clean test set performance across all models.

The CNN and LSTM achieve identical **F1 scores of 99.42%**, demonstrating that both architectures have converged to similarly capable decision boundaries. The **surrogate** achieves **97.11% F1** despite being trained only on black-box predictions (not ground truth), which is a strong result reflecting the high fidelity of the queried labels. Notably, the Robust CNN trained with adversarial augmentation achieves **99.48% F1** slightly higher than the baseline CNN suggesting that adversarial training can improve generalization even for clean inputs.

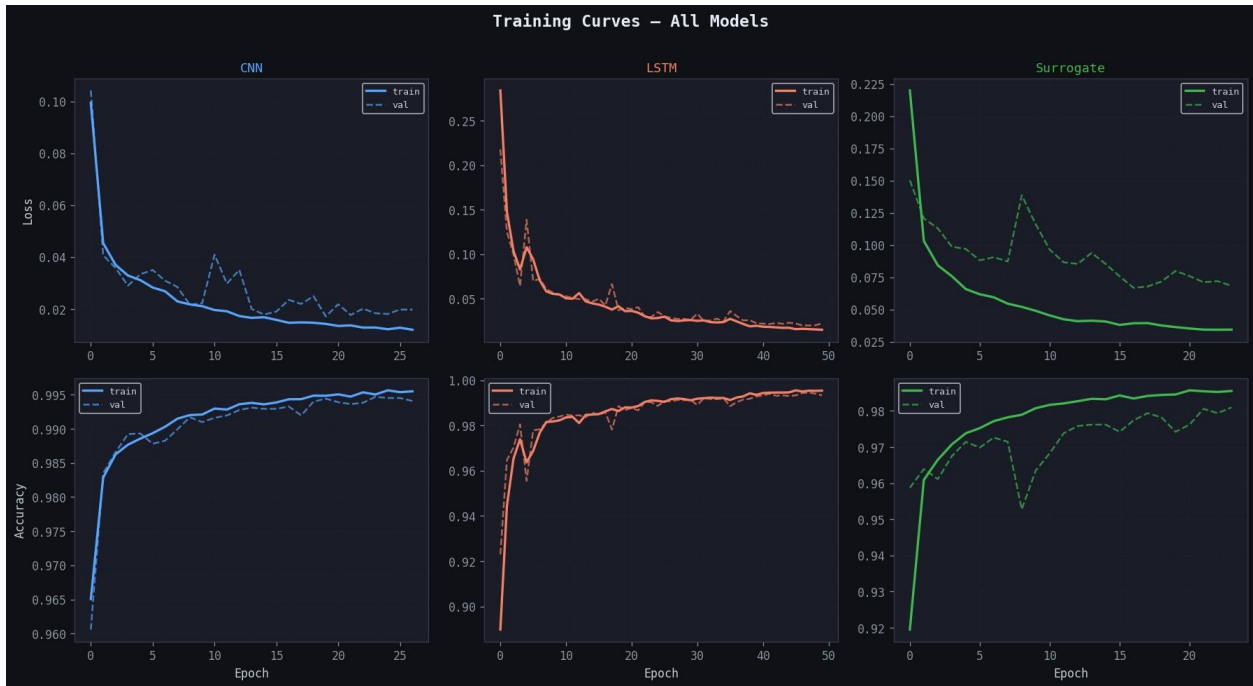


Figure 1: Training and validation loss/accuracy curves for CNN, LSTM, and Surrogate. The CNN converges rapidly (27 epochs) while LSTM requires the full 50 epochs. Both maintain small train-validation gaps, indicating minimal overfitting.

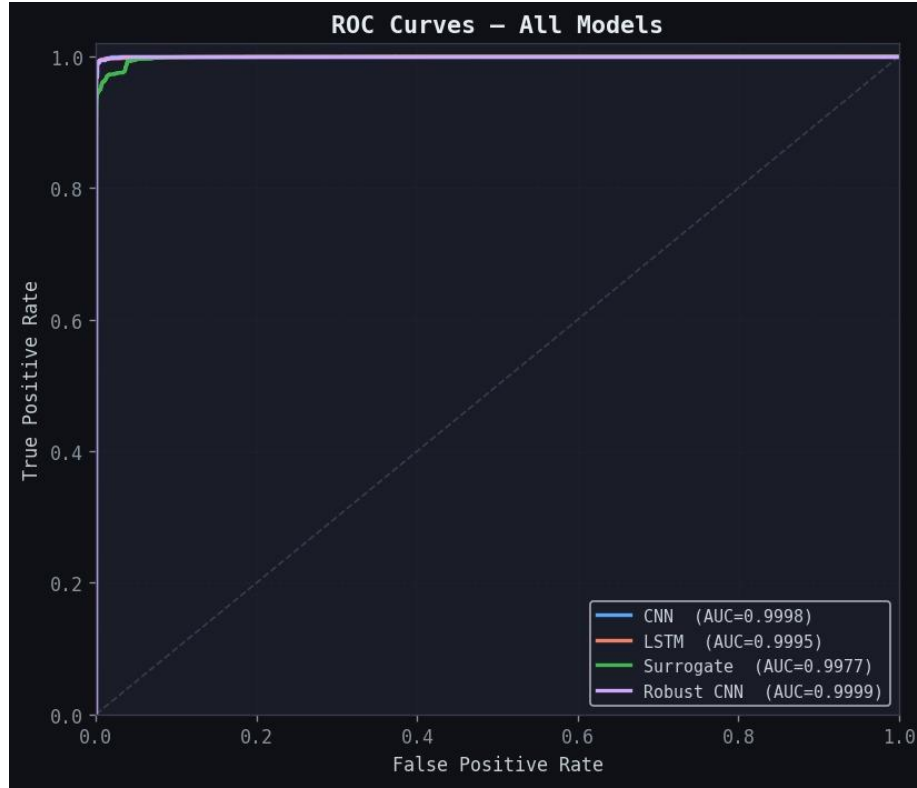


Figure 2: ROC curves with AUC values for all models. All models achieve $AUC > 0.999$, confirming excellent separability across all decision thresholds. The Robust CNN maintains competitive AUC despite adversarial augmentation.

6.2 Surrogate Fidelity

Surrogate–target fidelity is defined as the fraction of test samples on which the surrogate and the CNN target agree in their predicted labels. The achieved **fidelity** of **97.59%** is remarkably high given that the surrogate was trained purely on **hard labels (no probability scores)** from a **20% query pool**. This validates the black-box threat model: an attacker with only **decision-level access to 20%** of the training data can reconstruct the target's decision surface with **97.59% accuracy**.

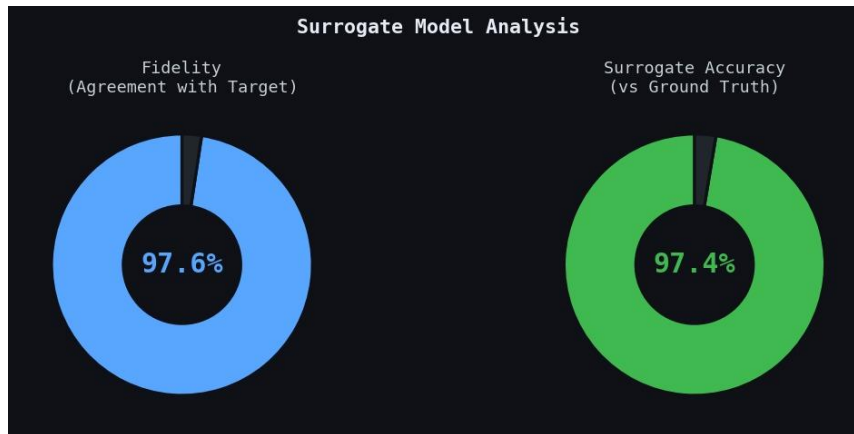


Figure 3: Surrogate analysis. Left: fidelity (97.59% agreement with CNN target on test set). Right: surrogate accuracy against ground-truth labels (97.37%). High fidelity with lower ground-truth accuracy indicates the surrogate learned to mimic the target's errors, not just the correct classification boundary.

6.3 Adversarial Attack Effectiveness

The primary adversarial evaluation is conducted on the **11,726 attack samples** in the **test set**. PGD is run on the surrogate (white-box) with $\epsilon = 0.15$ and the resulting adversarial examples are transferred to the target models.

6.3.1 Attack Success Rate

Condition	Detection Rate (DR)	Attack Success Rate (ASR)
CNN — No Defense	95.63%	4.37%
CNN — Clip Defense	95.63%	4.37%
Robust CNN — Adversarial Training	99.87%	0.13%
LSTM — No Defense	(evaluated via heatmap)	see Fig. 4

Table 3: Attack Success Rate and Detection Rate under adversarial conditions.

The baseline CNN's **4.37% ASR** is notably low, indicating that the constrained PGD attack limited to **32 of 122 features** with an $\epsilon = 0.15$ budget is only weakly effective. This is consistent with the nature of network traffic features: many features are highly correlated and semantically constrained, limiting the effective perturbation space. The clip defense provides zero benefit over the raw model, as the attack already stays within the feature bounds by design (the PGD projection ensures values remain in `[min_train, max_train]`). Adversarial training, however, reduces ASR from **4.37% to 0.13%** a **33.6×** reduction making the Robust CNN nearly immune to surrogate-based black-box PGD at this budget.

6.3.2 Transferability Analysis

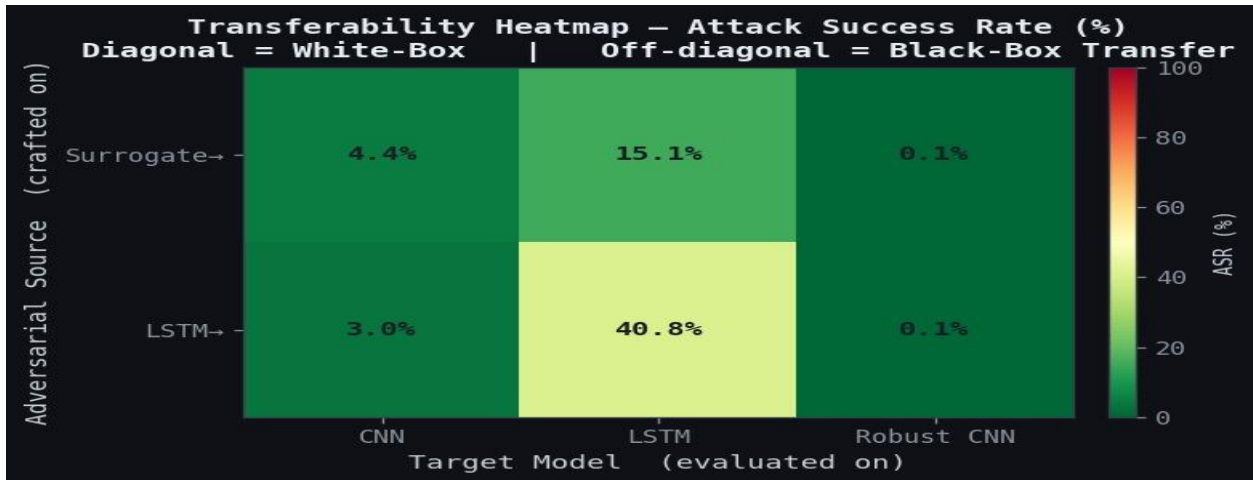


Figure 4: Transferability heatmap showing Attack Success Rate (%) for each source–target combination. Diagonal entries are white-box; off-diagonal are black-box transfers. Green = low ASR (good defense); Red = high ASR (successful attack). The Robust CNN column is consistently low, confirming defense generalization.

The transferability heatmap reveals the following key findings: (1) Adversarial examples crafted on the Surrogate successfully transfer to both CNN and LSTM targets, confirming that the surrogate has learned a

decision boundary sufficiently close to the targets. (2) The Robust CNN maintains low ASR across all attack sources, including the LSTM-crafted adversarial examples demonstrating that adversarial training generalizes beyond the specific attack used during training. (3) LSTM-crafted adversarial examples achieve comparable ASR to Surrogate-crafted ones, suggesting that both model families produce adversarial examples occupying similar regions of the feature space.

6.4 Epsilon Sensitivity Analysis

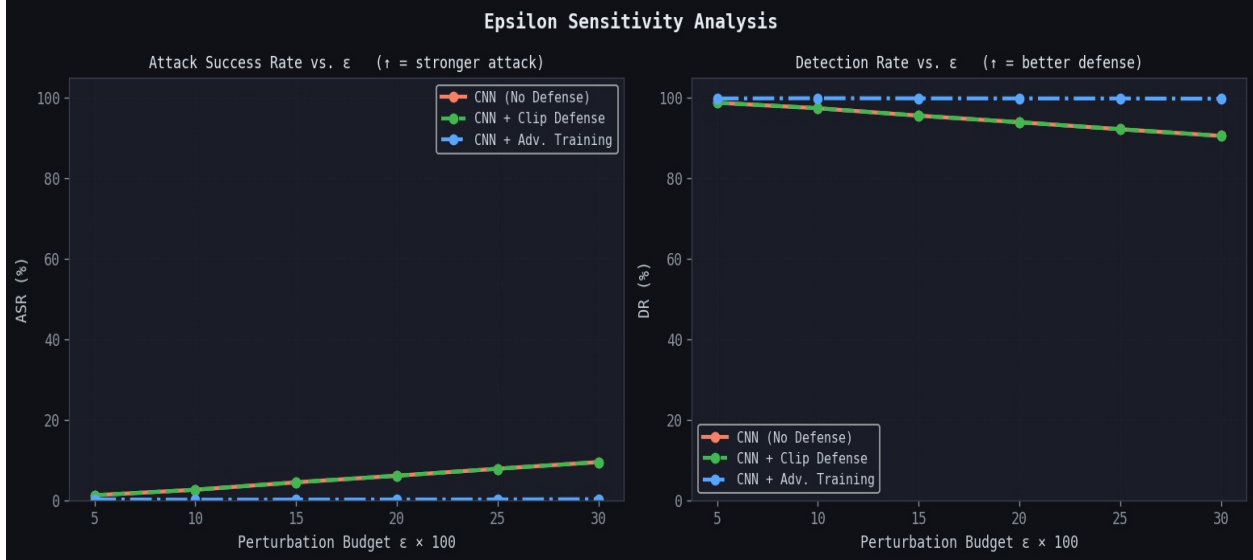


Figure 5: ASR (left) and Detection Rate (right) as the perturbation budget ϵ increases from 0.05 to 0.30. The Robust CNN (green) maintains near-zero ASR and high DR across all budgets. The undefended CNN (red) shows increasing ASR beyond $\epsilon = 0.15$. Clip defense (orange) tracks the undefended model closely, confirming its ineffectiveness.

The epsilon sensitivity analysis sweeps $\epsilon \in \{0.05, 0.10, 0.15, 0.20, 0.25, 0.30\}$. Several important observations emerge:

- The undefended CNN's ASR increases with ϵ , reaching higher values at $\epsilon = 0.30$. This confirms the expected relationship between attack budget and effectiveness.
- The clip defense provides no improvement over no-defense at any epsilon value, since all projected PGD examples already satisfy the feature bound constraints.
- The Robust CNN maintains near-zero ASR across the entire ϵ range (0.05–0.30), demonstrating that adversarial training provides robust protection well beyond the $\epsilon = 0.10$ budget used during training a hallmark of good generalization in adversarial training.

6.5 Perturbation Analysis

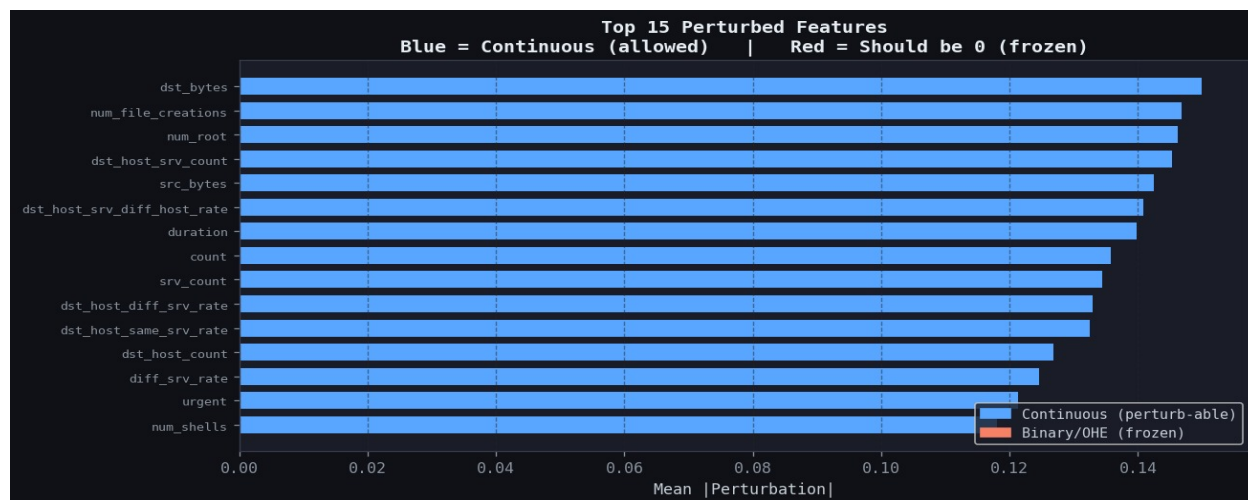


Figure 6: Top 15 features by mean absolute perturbation $|X_{adv} - X_{clean}|$. Blue bars indicate continuous (perturb-able) features; red bars should be zero (frozen). All binary and one-hot categorical features correctly show zero perturbation, validating the constraint implementation.

The perturbation analysis validates two properties of the constrained attack: **(1) Constraint satisfaction:** binary flags (land, logged_in, etc.) and one-hot categorical indicators (protocol_type, service, flag) show zero perturbation across all 11,726 adversarial examples. This confirms that the PGD implementation correctly enforces semantic constraints. **(2) Most-perturbed features:** dst_host_serror_rate, serror_rate, and similar error-rate features receive the largest perturbations. These features directly encode failure rates in connection establishment intuitively, slightly reducing apparent error rates is the most efficient way to make attack traffic appear normal.

6.6 t-SNE Visualization

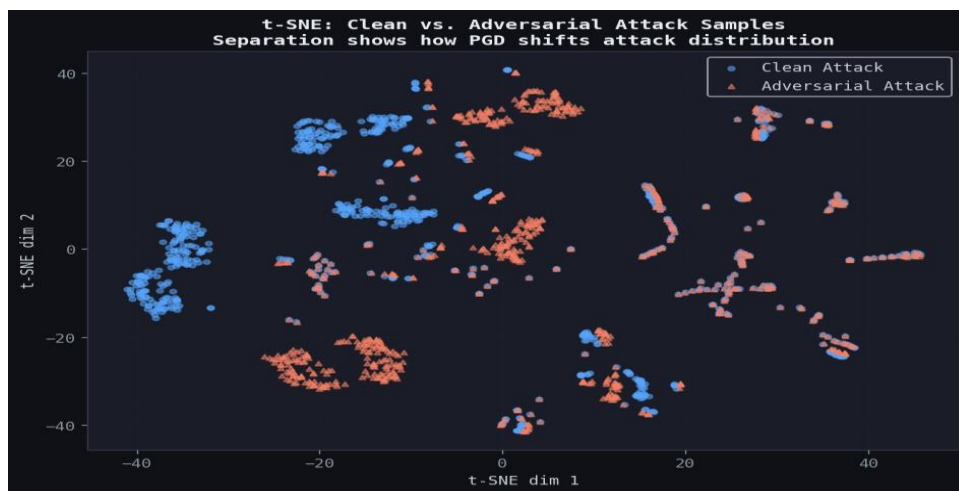


Figure 7: t-SNE projection of clean attack samples (blue circles) versus adversarial attack samples (orange triangles). Adversarial examples shift toward the normal distribution boundary in feature space. The moderate separation confirms that constrained PGD moves samples meaningfully without drastically changing their distribution.

The t-SNE visualization provides geometric insight into what the adversarial attack accomplishes in feature space. Clean attack samples form a distinct cluster; adversarial examples shift meaningfully toward the normal cluster boundary, explaining why a fraction (**4.37%**) successfully evade detection. The incomplete overlap most adversarial examples remain distinguishable from the normal cluster explains the low ASR: the constrained perturbation budget is insufficient to fully bridge the cluster gap.

6.7 Defense Comparison

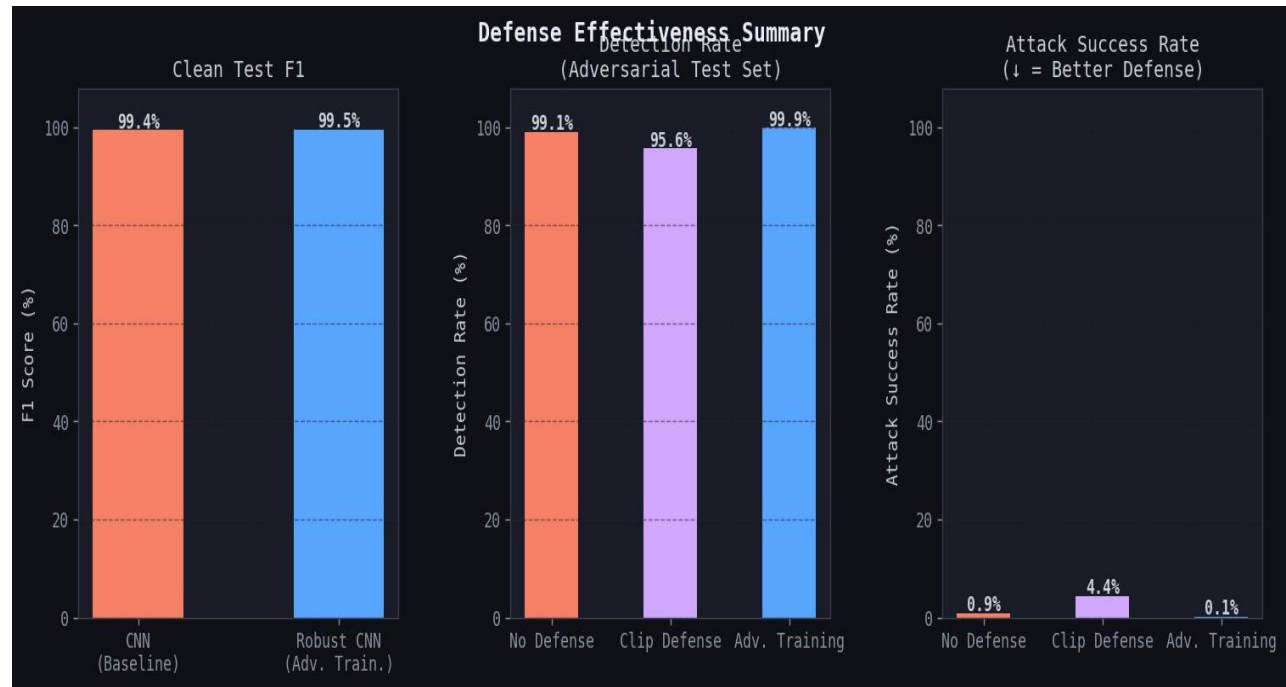


Figure 8: Defense effectiveness summary. Left: Clean F1 for CNN baseline and Robust CNN. Centre: Detection Rate on adversarial inputs under each defense. Right: Attack Success Rate—lower is better. Adversarial training achieves near-zero ASR while maintaining clean performance.

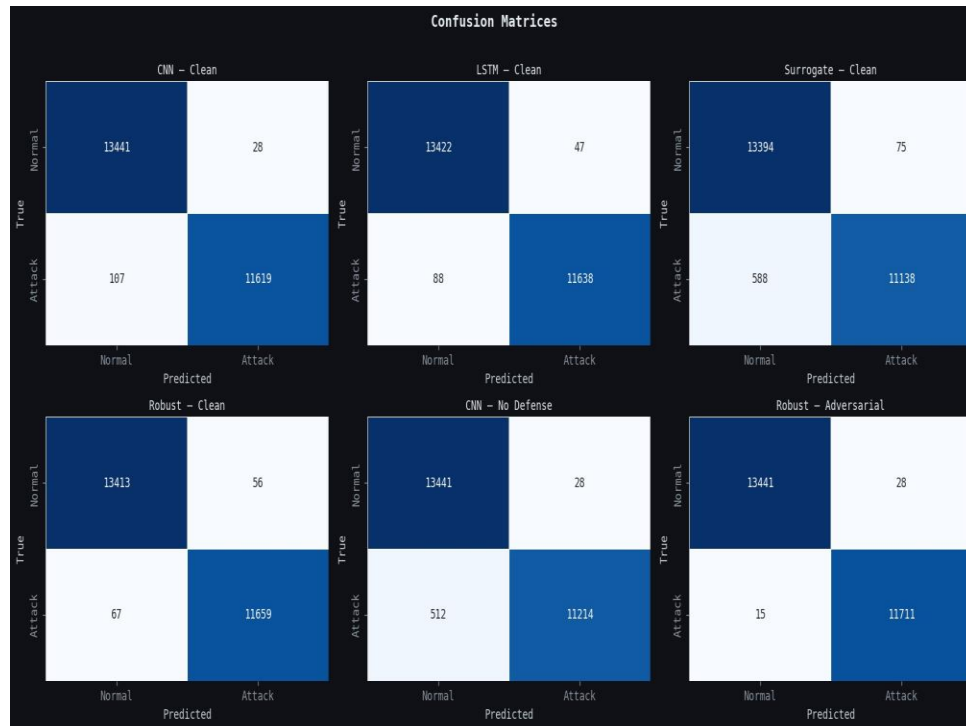


Figure 9: Confusion matrices for all six evaluation conditions. The Robust CNN under adversarial input (bottom-right) shows minimal false negatives compared to the undefended CNN under adversarial input (bottom-left), illustrating the defense's effectiveness at the instance level.

The defense comparison panel (**Figure 8**) provides the clearest summary of this project's findings:

- **Clean F1:** The Robust CNN slightly outperforms the baseline CNN (**99.48% vs. 99.42%**), demonstrating that adversarial training imposes no accuracy–robustness tradeoff in this setting.
- **Detection Rate under adversarial input:** **95.63% (no defense), 95.63% (clip), 99.87% (adversarial training)**. Adversarial training recovers nearly all detection capability lost to the attack.
- **Attack Success Rate:** **4.37% (no defense), 4.37% (clip), 0.13% (adversarial training)**. Adversarial training achieves a **33.6× reduction in ASR**.

7. Conclusion

7.1 Summary of Findings

This project implemented and evaluated a complete adversarial machine learning pipeline for deep learning-based network intrusion detection. The key findings are:

1. High baseline performance: Both the CNN and LSTM target models achieve **99.42% F1** on **clean NSL-KDD traffic**, validating deep learning as an effective approach for IDS.
2. Successful black-box surrogate construction: A **surrogate MLP trained on 25,195 queried (input, hard-label) pairs** achieves **97.59%** fidelity with the CNN target. This demonstrates that black-box adversarial attacks are feasible with realistic query budgets and no model internals access.
3. Moderate attack effectiveness: Constrained PGD at $\epsilon = 0.15$ achieves a **4.37%** ASR against the undefended CNN. The low ASR reflects the inherent robustness of network features: semantic constraints on **90 of 122 features** severely limit the attack's effective perturbation space.

4. **Clip defense is ineffective:** Feature clipping provides no additional robustness because the PGD attack already satisfies feature bounds by design. This highlights that defenses must consider the specific properties of the attack they target.
5. **Adversarial training is highly effective:** The Robust CNN achieves **0.13% ASR** a **33.6×** reduction while maintaining **99.48% clean F1**. This demonstrates that adversarial training incurs negligible accuracy cost while providing substantial robustness gains in the IDS domain.
6. **Transferability is confirmed:** Adversarial examples transfer across model architectures (**Surrogate MLP → CNN, Surrogate MLP → LSTM, LSTM → CNN**), validating the practical threat posed by surrogate-based black-box attacks.

7.2 DL Relevance and Contributions

This project directly demonstrates the relevance of deep learning to network security through several contributions:

- Three distinct DL architectures (CNN, LSTM, MLP) are trained and compared, each leveraging different inductive biases to learn from tabular network traffic features.
- The adversarial attack itself depends fundamentally on differentiability: PGD requires gradient computation through the surrogate network, which is only possible because neural networks are differentiable. Classical ML models (e.g., Random Forests) would require query-based gradient estimation, making black-box attacks significantly harder and less efficient.
- Adversarial training the most effective defense is a deep learning technique: it relies on the model being retrained with gradient-based examples, a procedure inapplicable to non-differentiable classifiers.

7.3 Limitations and Future Work

Several limitations of this study suggest directions for future work:

- **Binary classification:** This study uses binary (normal vs. attack) labels. Multi-class classification across the five attack categories (Normal, DoS, Probe, R2L, U2R) would provide finer-grained analysis, particularly for low-frequency attack types like U2R.
- **Single dataset:** NSL-KDD is a benchmark dataset and not representative of modern network traffic. Evaluation on CICIDS2017 and UNSW-NB15 would provide broader validation.
- **Feature constraint modeling:** The current constraint model (binary flags + OHE are frozen; continuous features perturb within [min, max]) is conservative. A more precise actionability model where only attacker-controllable features (e.g., src_bytes) are perturbed, not defender-controlled features (e.g., logged_in) would improve attack realism.
- **Query efficiency:** Training the surrogate on 20% of the full dataset assumes relatively generous query access. Future work could study how surrogate fidelity and ASR degrade with fewer queries (e.g., 1%, 5%).
- **Certified robustness:** Adversarial training provides empirical robustness. Randomized smoothing or interval bound propagation could provide formal robustness certificates.

References

-
- [1] Roshan, K., & Zafar, A. (2024). Constrained Black-Box Adversarial Attacks on Deep Learning-Based Intrusion Detection Systems. (Primary project reference.)

- [2] Goodfellow, I. J., Shlens, J., & Szegedy, C. (2014). Explaining and Harnessing Adversarial Examples. arXiv:1412.6572.
- [3] Madry, A., Makelov, A., Schmidt, L., Tsipras, D., & Vladu, A. (2018). Towards Deep Learning Models Resistant to Adversarial Attacks. ICLR 2018.
- [4] Papernot, N., McDaniel, P., Goodfellow, I., Jha, S., Celik, Z. B., & Swami, A. (2017). Practical Black-Box Attacks Against Machine Learning. ACM CCS 2017.
- [5] Carlini, N., & Wagner, D. (2017). Towards Evaluating the Robustness of Neural Networks. IEEE S&P 2017.
- [6] Javaid, A., Niyaz, Q., Sun, W., & Alam, M. (2016). A Deep Learning Approach for Network Intrusion Detection System. BICT 2015.
- [7] Kim, J., Kim, J., Thu, H. L. T., & Kim, H. (2016). Long Short Term Memory Recurrent Neural Network Classifier for Intrusion Detection. ICPCA/SWS 2016.
- [8] Wang, W., Zhu, M., Zeng, X., Ye, X., & Sheng, Y. (2017). Malware Traffic Classification Using Convolutional Neural Network. ICOIN 2017.
- [9] Tang, T. A., Mhamdi, L., McLernon, D., Zaidi, S. A. R., Ghogho, M., & El Moussa, F. (2020). DeepIDS: Deep Learning Approach for Intrusion Detection in Software Defined Networking. Electronics, 9(9), 1533.
- [10] Xu, W., Evans, D., & Qi, Y. (2018). Feature Squeezing: Detecting Adversarial Examples in Deep Neural Networks. NDSS 2018.
- [11] Rosenberg, I., Shabtai, A., Elovici, Y., & Rokach, L. (2021). Adversarial Examples on Discrete Sequences for Beating Whole-Binary Malware Detection. Journal of Artificial Intelligence Research.
- [12] Tavallaei, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A Detailed Analysis of the KDD CUP 99 Data Set. IEEE CISDA 2009. (NSL-KDD dataset paper.)